Trackback - Campus Lost & Found Web Application

Software Design & Architecture Report

Kyi Thant Sin (67011158) | May Nyein Chan (67011619) | Saw Zi Dunn (67011653)

---

1. Problem Description

Current State

Every semester, students and staff misplace personal items across campus - water bottles, ID cards, chargers, umbrellas, textbooks - while other people find these items with no easy way to return them. Today, this is handled informally through physical lost-and-found boxes at front desks, ad-hoc posts in campus social media groups, or word of mouth.

Problems with the current approach:

• Finders have no simple, structured way to report what they found

• Owners have no central place to search for a lost item

• There is no record of what has been turned in, where, or when

• Matching a "lost" report to a "found" report depends on luck, not the system

Proposed Solution

A lightweight web application where campus community members can:

• Report found items (title, description, category, photo, location, date/time)

• Report lost items (with the same metadata)

• Search and browse by category, keyword, location, or date

• See intelligent matches between lost and found items

• Claim found items or mark lost items as recovered

• Receive notifications when matches are found, an item owner requests to claim an item, or founder invites to do a claim request.

The system uses a passive Lost & Found Office (storage/pickup point) with no staff verification, removing overhead and requiring no physical infrastructure changes.

2. Requirements Model

Functional Requirements

User Accounts & Authentication

• Users can register with name, email, and password

• Users can log in securely (JWT in HTTP-only cookies)

• Users can update their profile (name, email, password, avatar)

Lost Items Management

• Users can report a lost item (title, description, category, last-seen location, date)

• Users can upload photos of the lost item

• Users can edit their own lost item posts

• Users can mark items as RECOVERED or delete them

• Lost Items have statuses: OPENED, RECOVERED, CLOSED

Found Items Management

• Finders can report a found item (title, description, category, location found, date)

• Finders can upload photos

• Found items have statuses: OPEN, PENDING_CLAIM, CLAIMED, CLOSED

Search & Browse

• Users can search by keyword (title, description, category, location)

• Users can filter by category, date range, and location

• Users can browse all posts (lost, found, or both)

Matching & Claims

• System **automatically** runs matching when items are posted

• Users can view candidate matches for their items via notifications

• Owners can claim a found item by submitting a claim request to a found-item post (submit justification + proof photo)

• Finders can approve or reject claim requests

• Finders can see lost-item posts and invite the owner to come and submit a claim request by linking to the found-item post

• On approval: claim status = APPROVED, found post = CLAIMED, lost item = RECOVERED

Notifications

• System notifies users when a match is found

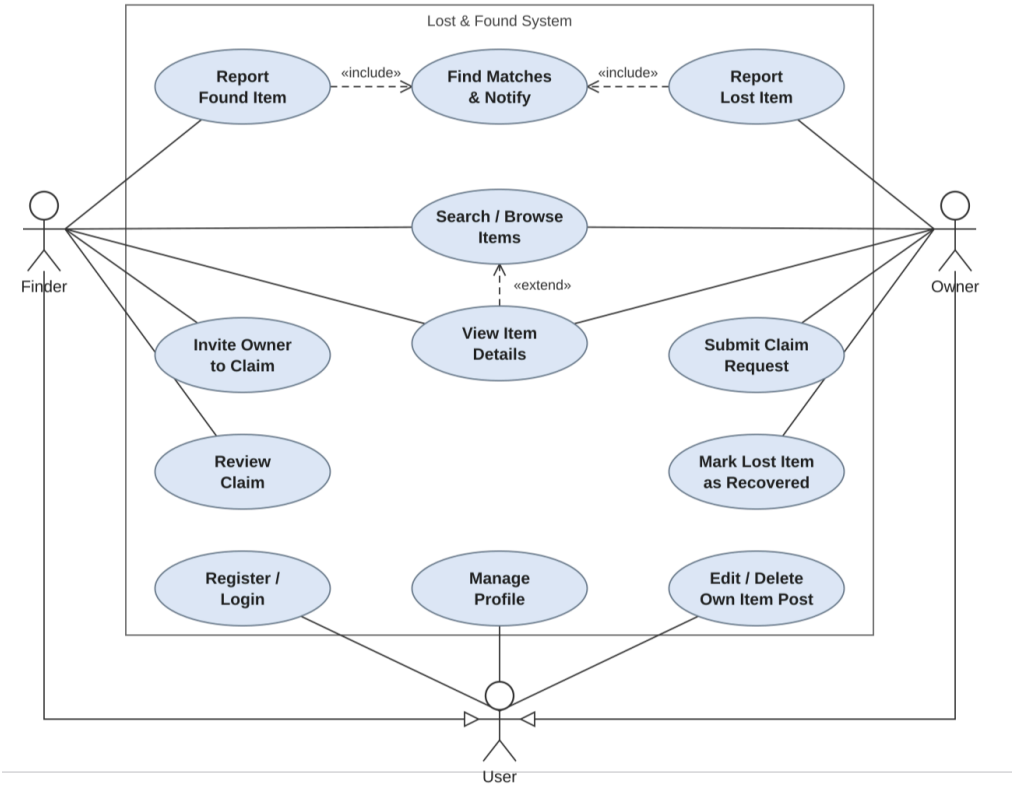
• System notifies finders when a claim is submitted

• System notifies owners when their claim is reviewed

• System notifies owner when finder invites to come and do a claim request to their found-item post.

• Users can mark notifications as read

Use Case Diagram



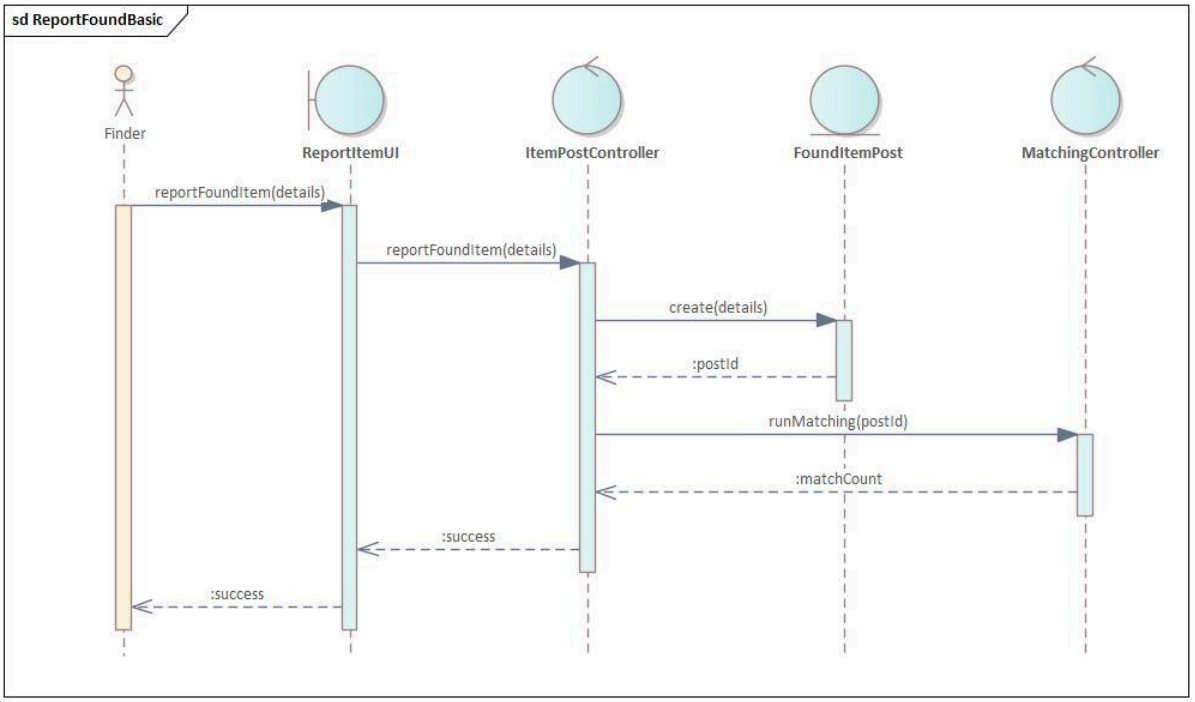
Use Case Description

Use Case	Description
Report found item	Finder submits item details (title, category, description, photo, location found, date). Includes Find matches & notify.
Report lost item	Owner submits item details (title, category, description, last-seen location, date). Includes Find matches & notify.
Find matches & notify	System compares the new post with open posts of the opposite type by category and keyword overlap, and sends an in-app notification to both sides when a likely match is found.
Search/browse items	Any user searches open posts by keyword, category, location, or date.
View item details	User opens a post to see its full details and photo. Extends Search/browse items.
Invite owner to claim	Finder invites the owner of a lost post to claim a found item they are holding. The owner is notified and can do claim request on the found-item post.

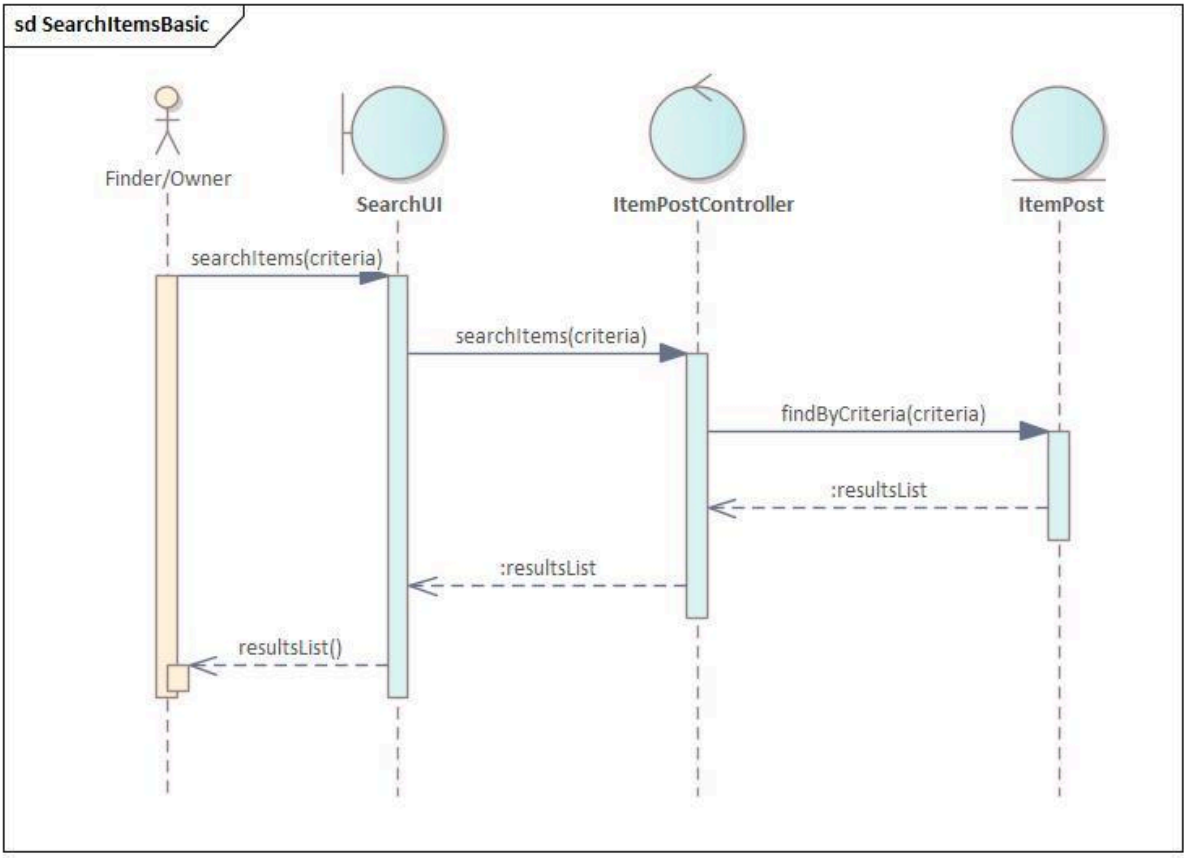
Submit claim request	Owner submits a claim on a found post with a justification, item details, and an optional proof photo. The finder is notified.
Review claim	Finder approves or rejects a claim. Approval marks the found item as claimed and the owner's lost post as recovered. Rejection reopens the found post.
Mark lost item recovered	Owner closes their own lost-item post when the item is recovered outside the claim flow.
Register / login	User creates an account with a campus email, or signs in to an existing account.
Manage profile	User updates their name, email, password, about text, and avatar.
Edit / delete own item post	User edits or deletes a post they created. Lost posts can be edited only while they are open.

Sequence Diagrams

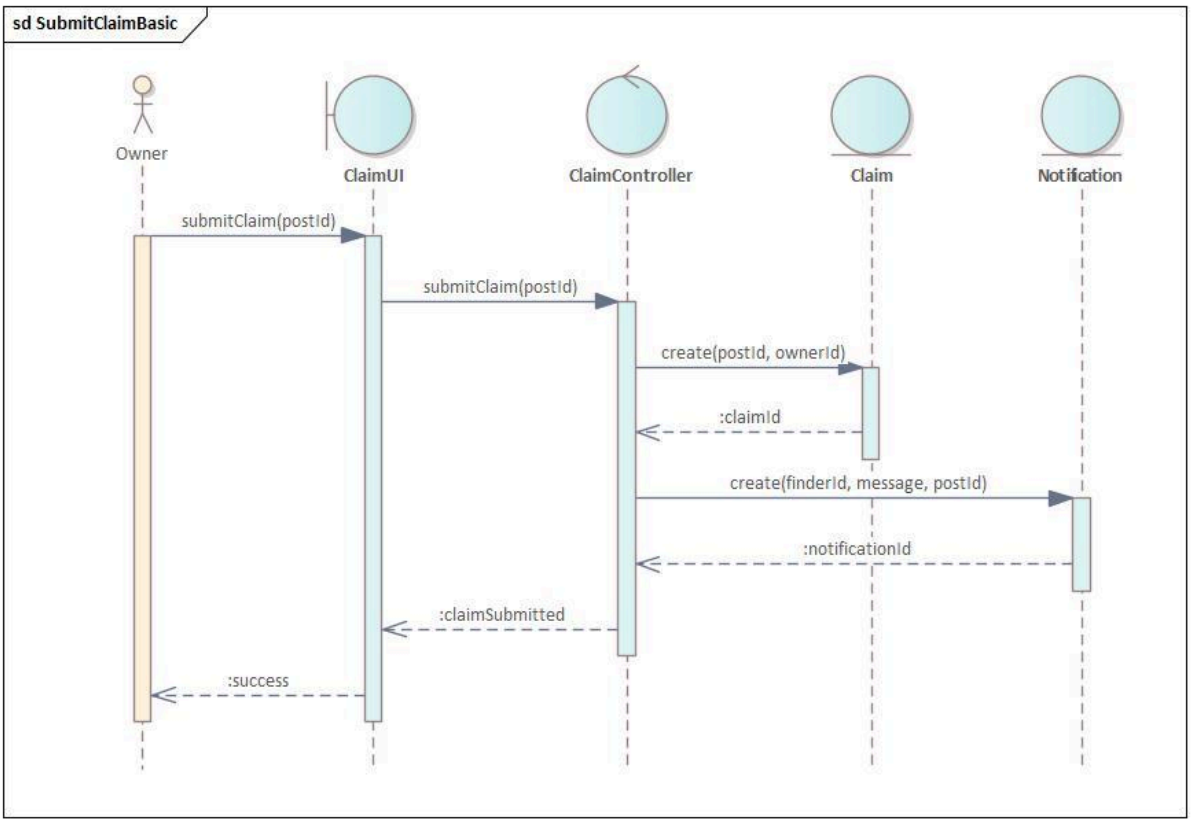
1. Report Found Item



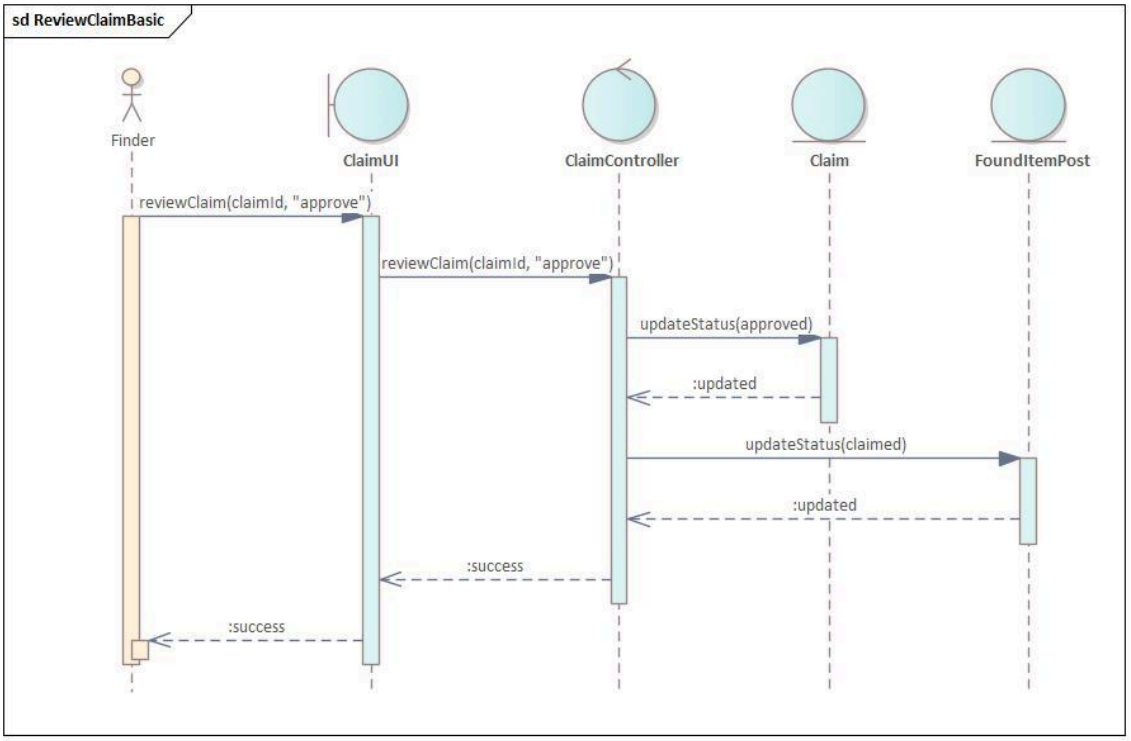
2. Search Items



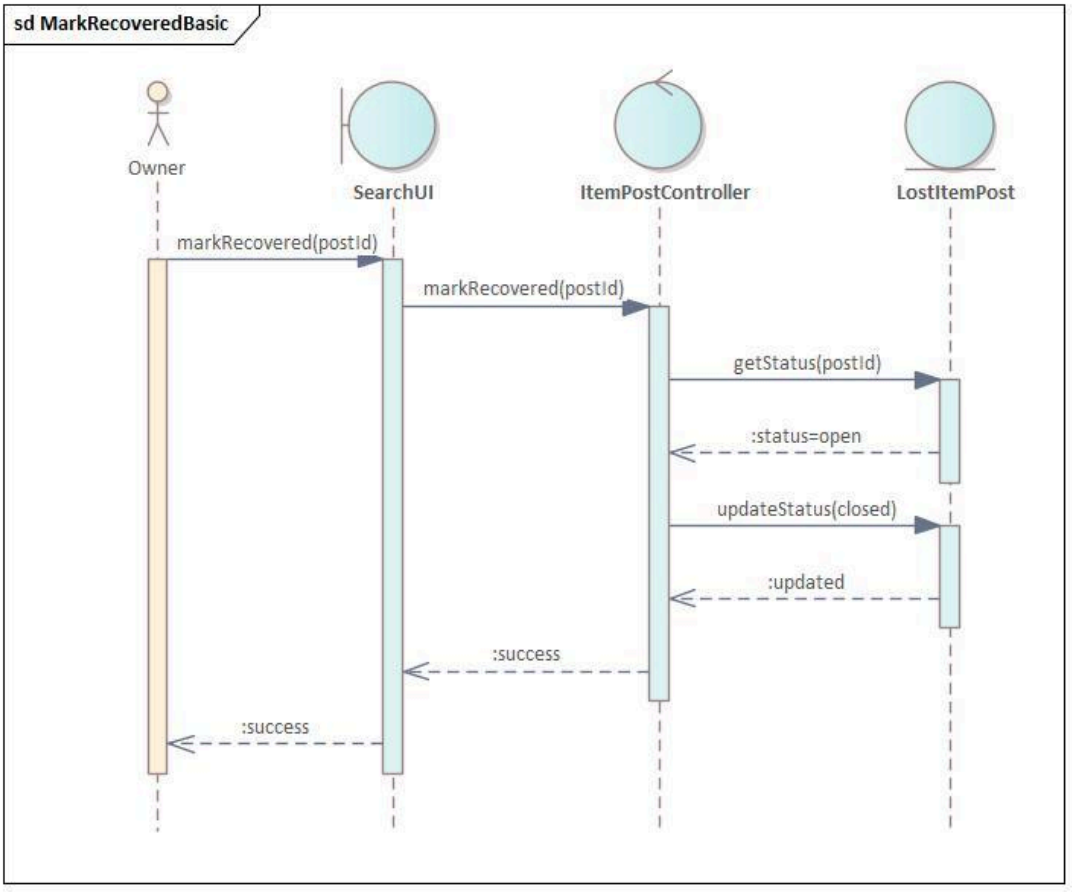
3. Submit Claim



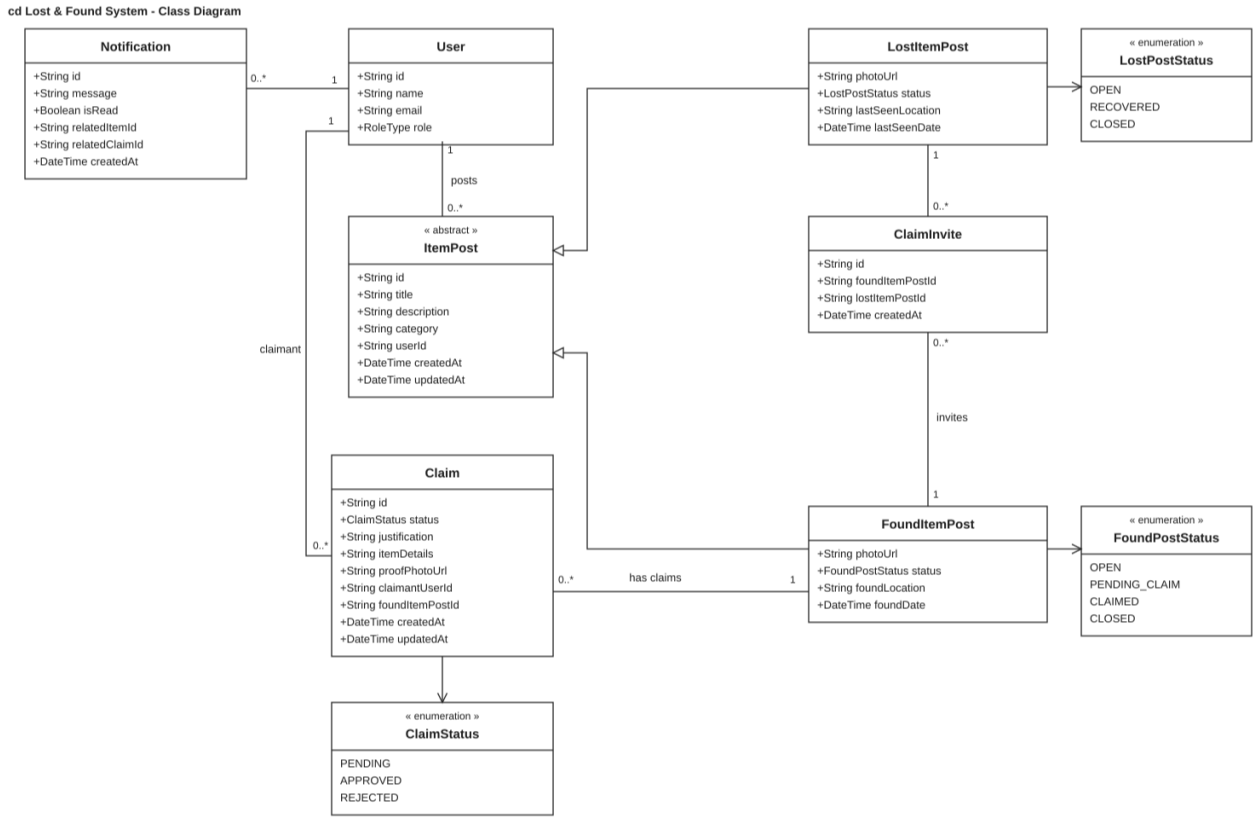
4. Review Claim



5. Mark Item as Recovered

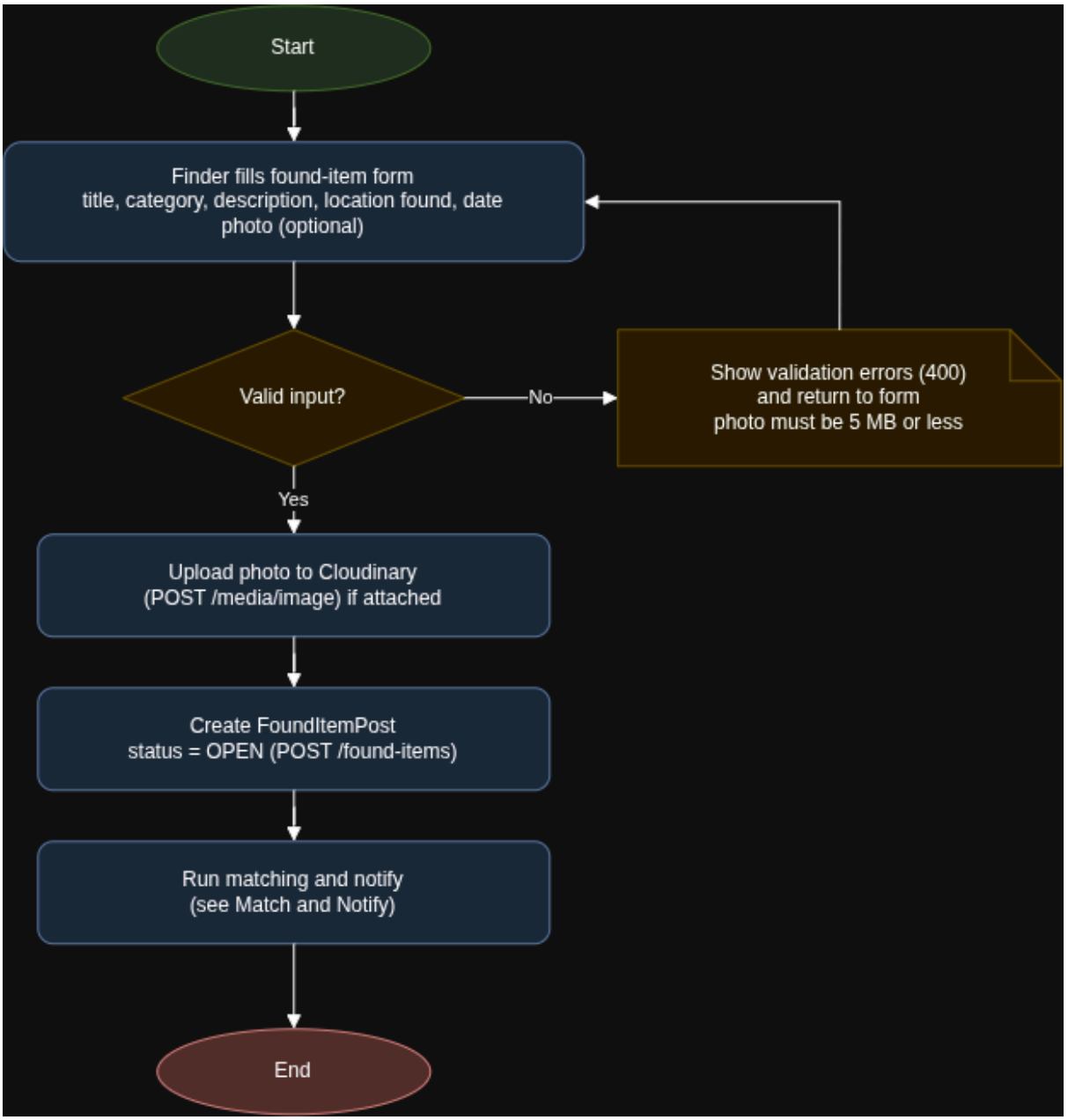


Domain Class Diagrams

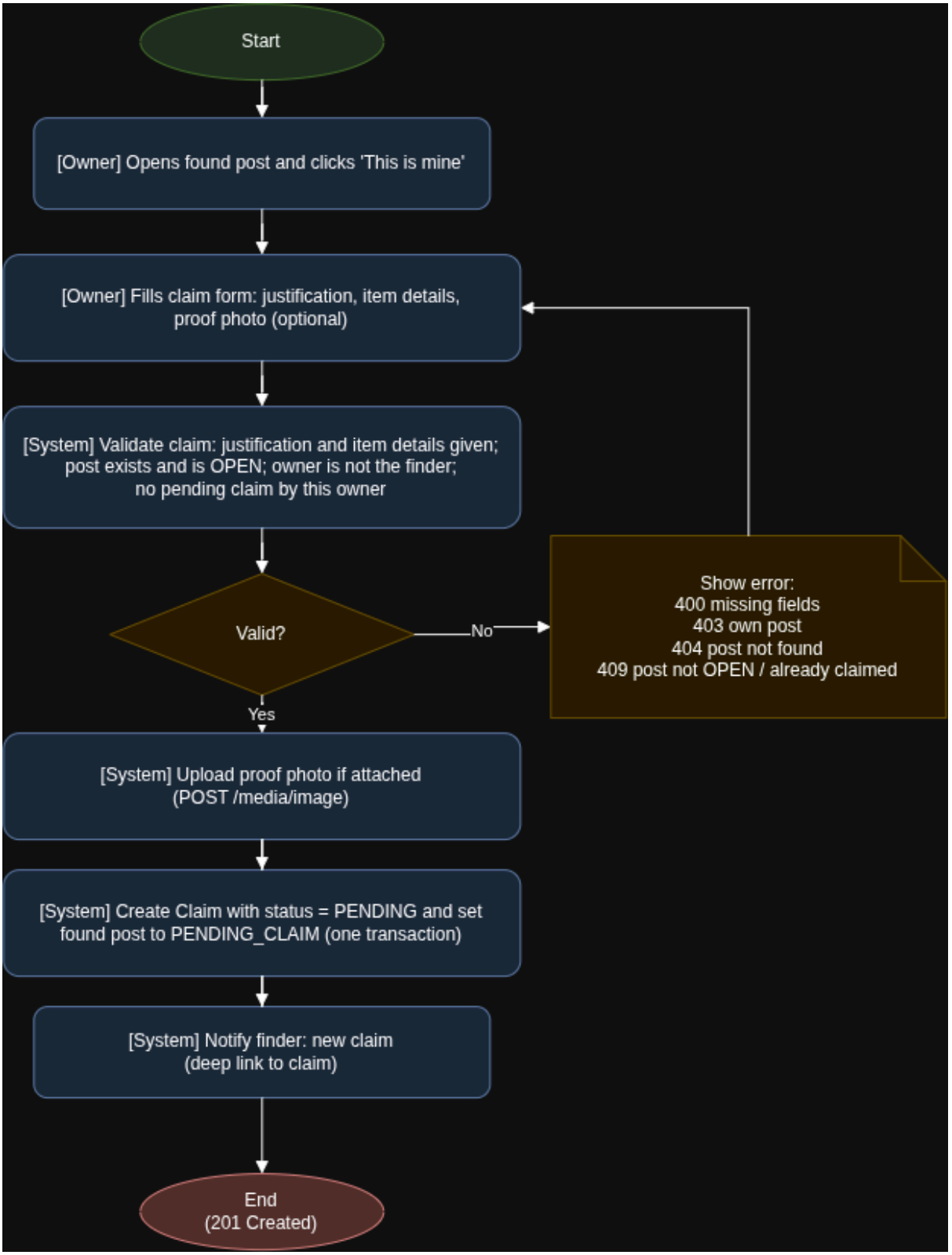


Activity Diagrams

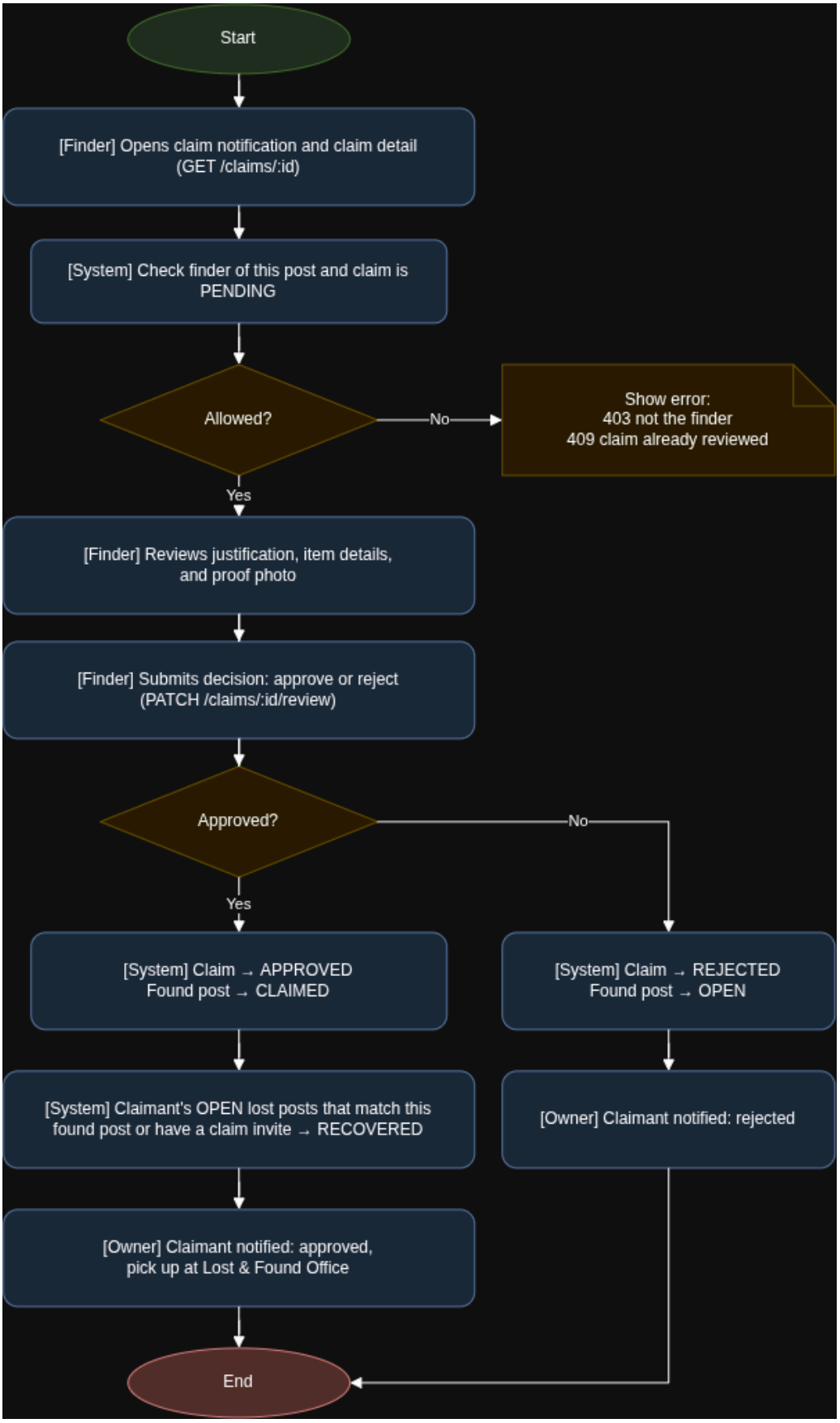
1. Report Found Item



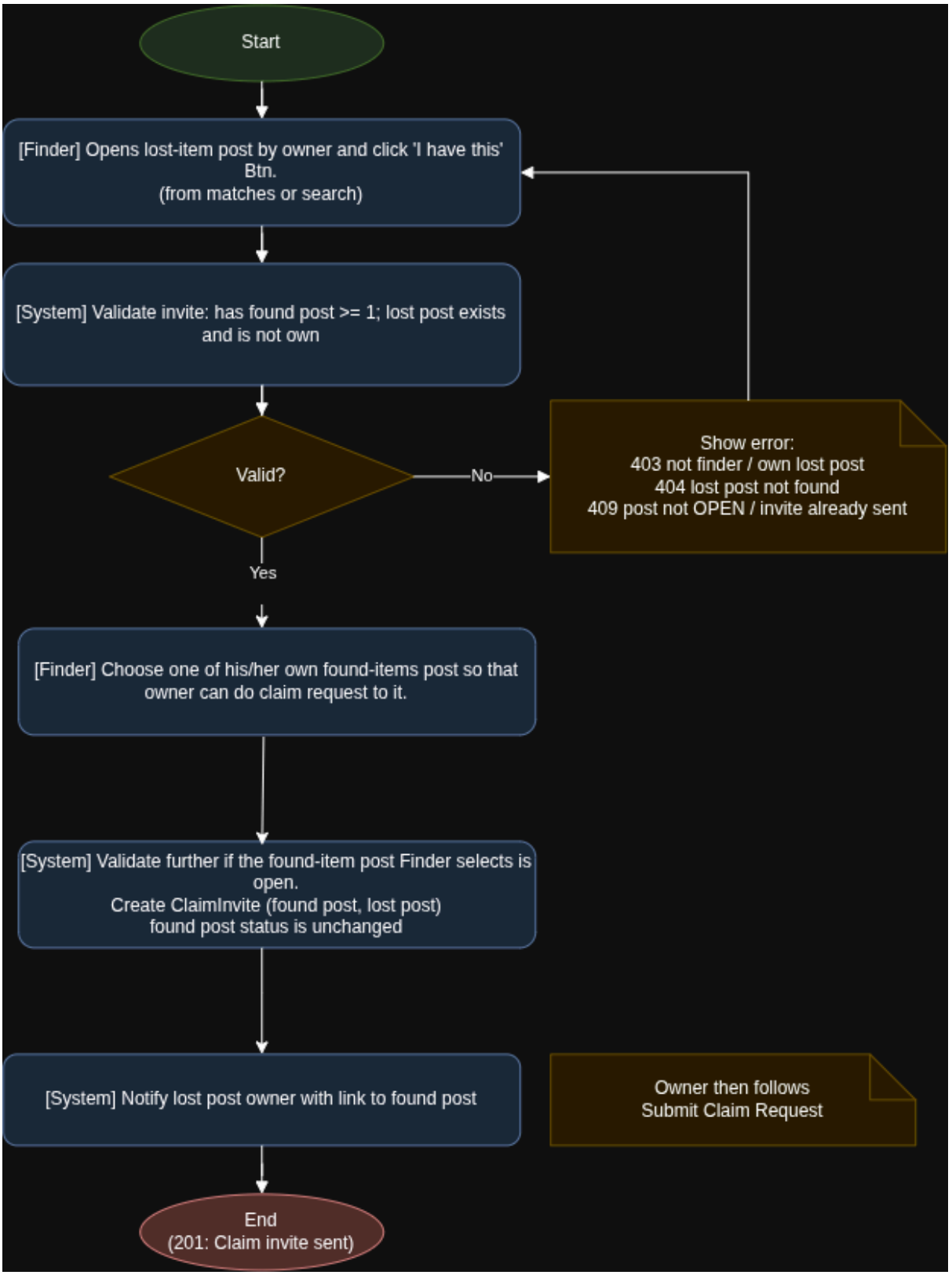
2. Submit Claim Request



3. Review Claim Request



4. Invite owner for claim request



Non-Functional Requirements

Requirement	Target / measure	Current design support
Performance	Response times to be agreed and verified by testing	List endpoints are paginated (default 10 items per page, maximum 100)
Security	Only logged-in users can use the system; passwords are stored hashed; uploads are limited to 5 MB per file	JWT in an HTTP-only cookie (1 day, or 30 days with remember-me); owner checks on every edit and delete
Usability	Reporting, searching, and claiming each take a small number of steps	One form per report; search with filters; claiming is a single form. Verify with user testing
Scalability	Expected number of concurrent users to be agreed and tested	Single deployable application (see Architectural Characteristics)
Maintainability	A change to one feature does not require changes to unrelated features	Modular structure, layered backend, shared design patterns

3. Design

System Architecture Overview

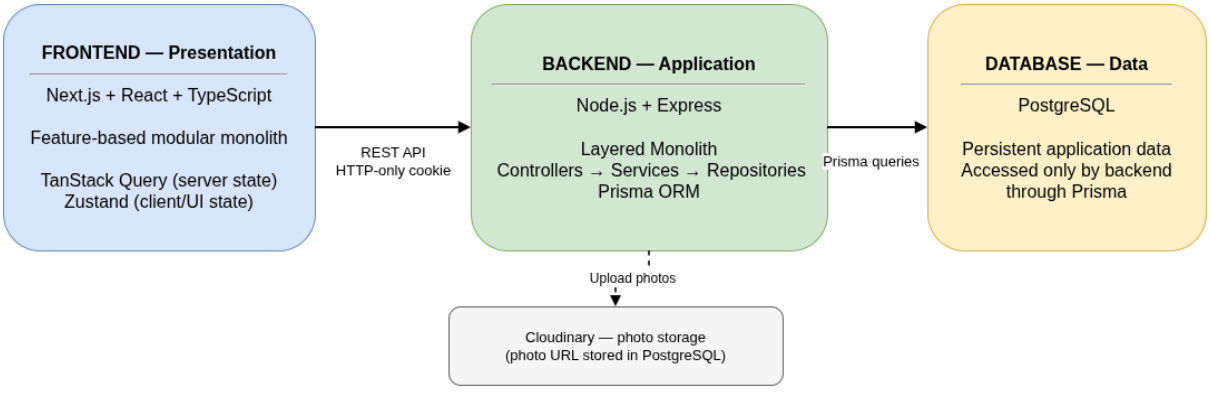
TrackBack is a **three-tier client-server** web application:

• Frontend (**Presentation**): Next.js + React + TypeScript (feature-based modular monolith)

• Backend (**Application**): Node.js + Express (layered monolith architecture with controllers, services, repositories)

• Database (**Data**): PostgreSQL - Persistent storage; accessed only through Prisma from tier-2

The frontend and backend communicate via a REST API with HTTP-only cookie authentication. Photos are stored on Cloudinary.



Frontend Architecture

The frontend uses a **feature-based modular monolith** architecture. Each feature module contains all its own pages, components, API hooks, and types.

Folder Structure:

src/app/ (routing with Next.js App Router)

src/modules/ **(auth, lost-items, found-items, claims, notifications, profile)**

src/core/ (API client, providers, global state)

src/common/ (shared components)

Key Design Principles:

• One-way dependencies: modules can use core/ and common/; core/ never imports from modules/
• TanStack Query owns server state (items, claims, notifications)
• Zustand owns client session/UI state
• Every API hook is a facade hiding HTTP, caching, and invalidation

Backend Architecture

Backend uses a **Layered monolith** architecture.

Request → Route → Middleware → **Controller** → **Service** → **Repository** → Database

Folder Structure:

src/routes - API/Interface layer; defines API endpoints and router composition.

src/controllers - API layer; reads and validates HTTP requests, calls services, and formats HTTP responses.

src/middlewares - API layer; handles authentication, file uploads, and centralized error handling.

src/services - Business logic layer; implements business rules, use cases, and workflow coordination.

src/repositories - Data-access layer; handles database queries and persistence.

src/integrations - Infrastructure support; handles communication with external providers such as Cloudinary and SMTP email.

src/app.ts - Application entry point; creates and configures Express, registers middleware and routes, and starts the server.

Design Patterns Applied

Frontend: Singleton, Facade

Backend: Chain of Responsibility, Facade, Adapter, Factory, Strategy, Singleton

So TrackBack supports **SOLID** principles. Strategy and Chain of Responsibility make it easier to add new behavior (OCP). Facade and Adapter help separate responsibilities (SRP). Small, focused interfaces support ISP, implementations that can replace one another without breaking expected behavior support LSP, and depending on abstractions keeps business logic independent of external tools (DIP).

Matching Algorithm

Rule: Same category (case-insensitive) AND ≥1 shared word (≥3 chars) in title + description.

Score = 0.6 + 0.4 × (overlap ratio); threshold ≥ 0.6. Only OPEN posts. Matches computed live on item creation.

Architectural Characteristics

Characteristic	Where it applies	How the architecture supports it	Evidence in the design	Limitation
Modularity	Both	Features are grouped into separate modules (lost items, found items, claims, notifications, matching)	Feature modules on the frontend; route, controller, service, and repository per feature on the backend	Module boundaries are kept by convention and code review, not enforced by tooling
Coupling	Both	Layers and one-way dependencies limit what each part knows about the others	Backend layers; frontend rule that `core/` never imports from `modules/`	Coupling is reduced, not eliminated; matching and claims depend on several modules
Cohesion	Both	Each module holds closely related responsibilities	Lost-item service contains lost-item use cases; Cloudinary integration contains image operations	Claims and matching intentionally coordinate several features
Maintainability	Both	Business rules live in services, not in route handlers; API hooks hide HTTP details from components	Service layer on the backend; facade hooks on the frontend	Rules must stay in sync with the API reference when they change
Extensibility	Both	New endpoints, use cases, repositories, or providers can be added without changing unrelated layers	Strategy-style matching seam (the v1 rule can be replaced); Adapter for Cloudinary	Adding a new entity still touches routes, types, and documentation
Portability	Backend	Provider-specific code is isolated in repositories and integrations	Cloudinary Facade and Adapter; Prisma repositories	Database schema changes still touch repositories and types

Reliability	Backend	Shared middleware and central error handling give consistent authentication, upload validation, and error responses	`secureRoute` middleware; upload size check; central error handler	Calls to Cloudinary and SMTP are not retried or queued in the current design
Scalability and elasticity	Backend	Clear boundaries make it possible to optimize or extract expensive parts later	Matching is isolated in one module	A single Express process; no horizontal scaling, queues, or independent deployment yet
Security	Both	Authentication and request checks are applied through shared middleware, not per endpoint; the frontend guards protected pages	`secureRoute`; owner checks in services; hashed passwords; HTTP-only cookies; frontend `AuthGuard`	Password reset accepts passwords of at least 4 characters, which is weak; confirm HTTPS at deployment

4. Implementation

Technology Stack

Layer	Technology
Frontend	Next.js 14, React 18, TypeScript, TanStack Query, Zustand, Tailwind CSS, Axios
Backend	Node.js, Express, TypeScript, Prisma ORM
Database	PostgreSQL
Authentication	JWT in HTTP-only cookies
Image Storage	Cloudinary

Frontend Code Organization

Each feature module contains pages/, components/, api/, and types/. The api/ layer uses TanStack Query hooks (facades) wrapping HTTP calls, caching, and invalidation. Components import hooks and use them without knowing about Axios.

Example: useSubmitClaim() handles form submission, proof upload, error handling, and query invalidation in one hook.

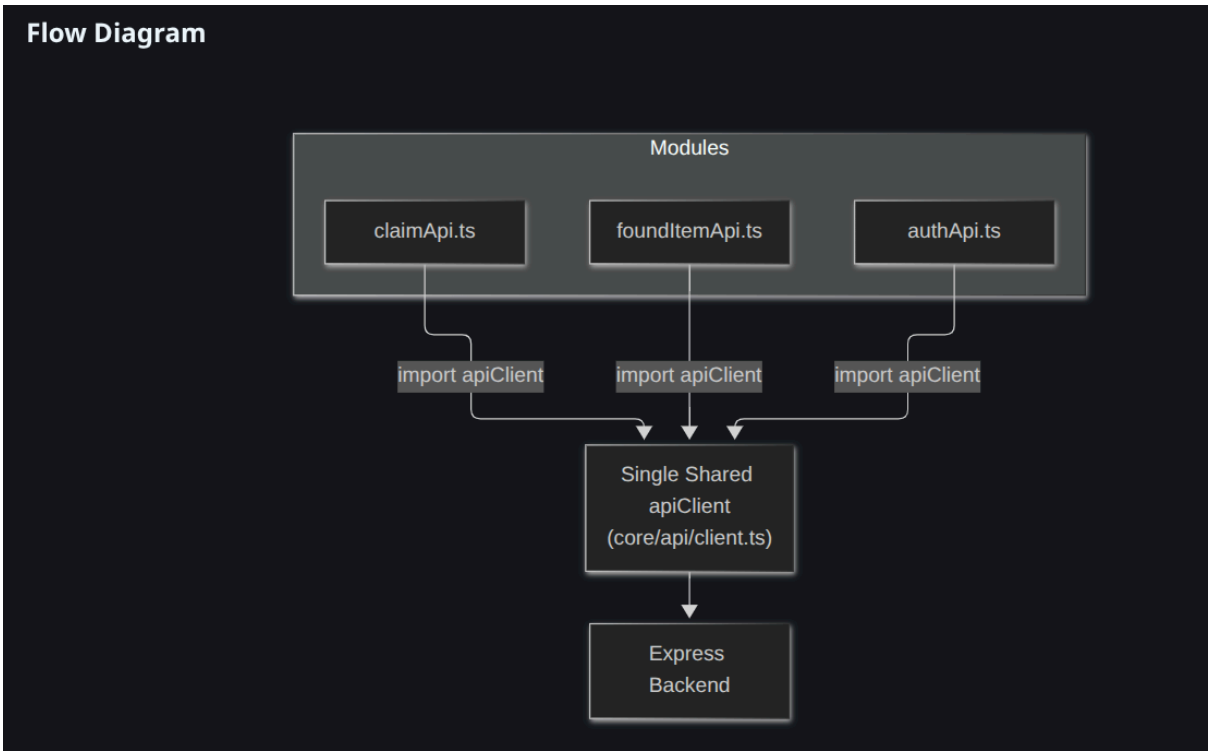
Design Patterns Applied in Frontend

1. Singleton Pattern (Creational Design Pattern)

Core Idea: Create exactly one instance of an object and share it across the whole application.

The Problem Without It: If every feature created its own Axios instance, the backend base URL, withCredentials, and interceptors would be configured in a dozen places. A 401 handler would fire per instance, and changing the API URL (development vs. deployed) would mean editing every module.

How TrackBack Implements It: src/core/api/client.ts creates one configured Axios instance. Because ES(ECMA Script) Modules are cached after the first import, every API hook in every module receives the same apiClient. The same approach gives us one QueryClient configuration (src/core/api/queryClient.ts) and one Zustand store (src/core/store/useAppStore.ts).



Code Implementation (`src/core/api/client.ts`):

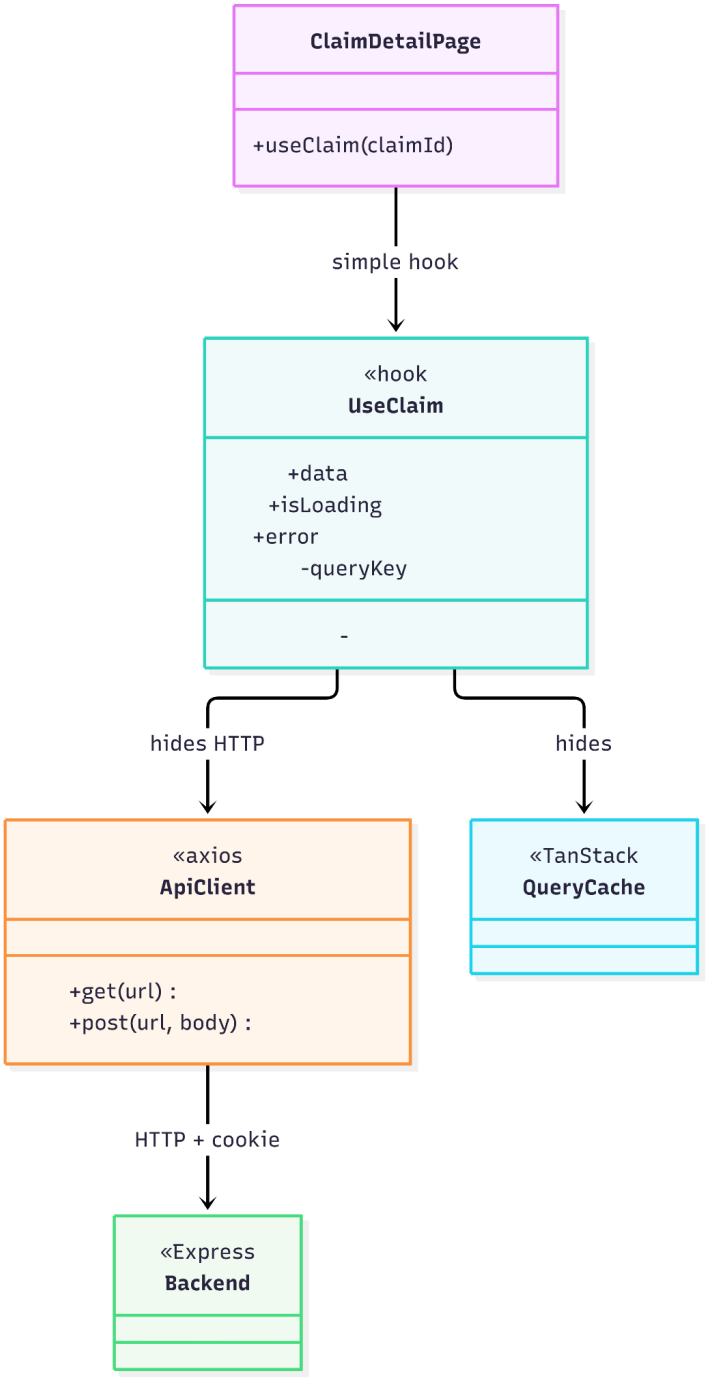
```
src > core > api > TS client.ts > ...
1   import axios from "axios";
2   import { setupInterceptors } from "./interceptors";
3
4   // SINGLETON: one configured axios instance shared by every feature module
5   export const apiClient = axios.create({
6     baseURL: process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8080",
7     withCredentials: true,
8     headers: {
9       "Content-Type": "application/json",
10    },
11  });
12
13  setupInterceptors(apiClient);
```

2. Facade Pattern (Structural Pattern)

Core Idea: Hide implementation details behind a clean, high-level interface.

The Problem Without It: Components would call Axios directly, manage isLoading and error flags by hand, invent their own cache keys, and remember to invalidate the right queries after every mutation. That boilerplate would be copied into every page.

How TrackBack Implements It: Each feature's api/ layer (for example, src/modules/claims/api/claimApi.ts) wraps the whole data flow (HTTP call, response typing, query key, caching, and invalidation) in one custom hook. Components call useClaims() or useClaim(id) and read { data, isLoading, error }. They never touch Axios directly.



Code Implementation (src/modules/claims/api/claimApi.ts, simplified):

```
src > modules > claims > api > TS claimApi.ts > useClaims > queryFn > response
64  // List claims for the current user — as claimant (mine) or as finder (toReview)
65  export function useClaims(
66    role: ClaimRole,
67    status: ClaimStatus | "",
68    page: number,
69    options?: { enabled?: boolean },
70  ) {
71    return useQuery({
72      queryKey: [...CLAIMS_QUERY_KEY, role, status, page],
73      queryFn: async (): Promise<ClaimsResult> => {
74        const response = await apiClient.get<ClaimsResult>("/claims", {
75          params: {
76            role,
77            ...(status ? { status } : {}),
78            page,
79            limit: PAGE_SIZE,
80          },
81        });
82        return response.data;
83      },
84      enabled: options?.enabled ?? true,
85    });
86  }
87
```

Backend Code Organization

Flow: Route → Controller → Service → Repository → Database

• Routes define endpoints and middleware chain

• Controllers translate HTTP requests to service calls

• Services contain business logic and use cases

• Repositories isolate database queries

Example: POST /found-items/:id/claims uploads proof to Cloudinary, creates claim, notifies finder, updates post status—all in the service layer.

Design Patterns Applied in Backend

1. Chain of Responsibility Pattern (Behavioral Pattern)

Core Idea: Pass a request through a chain of handlers. Each handler either does its job and passes the request to the next worker (next()), or stops the line immediately if something is wrong.

How TrackBack Implements It:

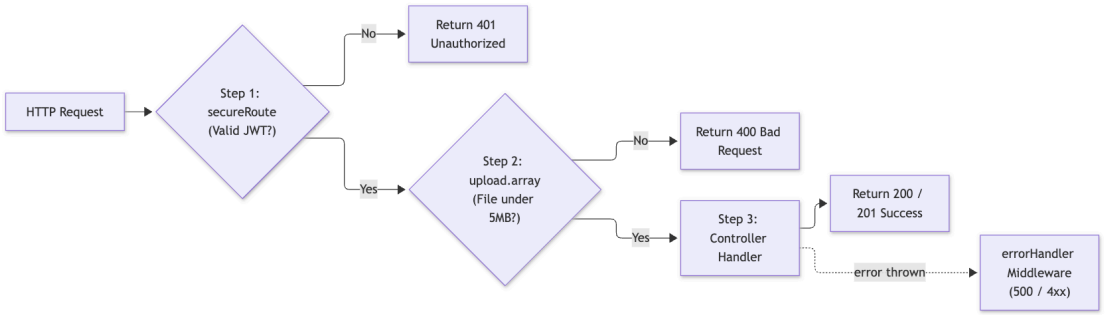
In the Express middleware pipeline:

1. `secureRoute`: Checks the authentication cookie and rejects with 401 if invalid.

2. `upload.array("image")`: Parses file uploads and rejects with 400 if the file is too large.

3. `controller`: Executes the requested action.

4. `errorHandler`: Centralized error safety net that catches any thrown exceptions.



Code Implementation:

Pipeline in Route (`src/routes/media/media.ts`):

```
router.post(
  "/image",
  secureRoute,              // 1. Authenticate user
  upload.array("image"),    // 2. Validate and process upload
  uploadController.uploadImage // 3. Run controller
);
```

Guard Worker (`src/middlewares/secure-route.ts`):

```
const secureRoute = (req: Request, _res: Response, next: NextFunction) => {
  const token = req.cookies?.token;
  if (!token) {
    return next(new AuthenticationError("Authentication required")); // Stop chain
  }
  try {
    const payload = jwt.verify(token, process.env.ACCESS_TOKEN_SECRET!);
    req.userId = (payload as any).userId;
    return next(); // Pass to next middleware
  } catch {
    return next(new AuthenticationError("Invalid or expired access token"));
  }
};
```

2. Facade Pattern (Structural Pattern)

Core Idea: Hide implementation details behind a clean, high-level interface.

How TrackBack Implements It:

1. Routing Facade (src/routes/gateway.ts): Unifies 10 separate module routers (`auth`, `users`, `lost-items`, `found-items`, `claims`, `notifications`, and others) into a single `gateway` router.

2. Cloudinary Facade (`src/integrations/cloudinary/cloudinaryService.ts`): Hides complex SDK streaming configuration behind friendly functions (`uploadImage`, `deleteImage`).

```
const gateway = Router();

gateway.use('/auth', authRouter);
gateway.use('/users', userRouter);
gateway.use('/', claimsRouter);
gateway.use('/found-items', foundItemsRouter);
gateway.use('/lost-items', lostItemsRouter);
gateway.use('/', notificationsRouter);
gateway.use('/media', mediaRouter);
gateway.use('/', matchingRouter);
gateway.use('/items', browseItemRouter);      You, last week via  PR #16
gateway.use('/my-posts', myPostsRouter);

export default gateway;
```

3. Adapter Pattern (Structural Pattern)

Core Idea: Convert one interface into another that the caller expects, so incompatible pieces can work together.

How TrackBack Implements It:

`src/integrations/cloudinary/cloudinaryService.ts` acts as an Adapter. It takes Cloudinary's callback-based stream API and adapts it into a standard JavaScript `Promise<UploadedImage>`.

```
// ADAPTER: Converts callback stream into a clean async/await Promise
const uploadImage = (
  file: Buffer,
  folder: CloudinaryFolder,
): Promise<UploadedImage> =>
  new Promise((resolve, reject) => {
    // 3rd-party callback/stream interface
    cloudinary.uploader
      .upload_stream(
        {
          folder: `${cloudinaryRootFolder}/${folder}`,
          resource_type: "image",
        },
        (error, result) => {
          if (error || !result)
            return reject(error ?? new Error("Upload failed"));
          // Adapts 3rd-party shape into TrackBack's UploadedImage type
          resolve({ url: result.secure_url, publicId: result.public_id });
        },
      )
      .end(file); // pipes buffer into stream
  });
```

4. Factory Pattern (Creational Pattern)

The Problem Without It: Every email needs the same HTML boilerplate: a background, a white card, the TrackBack heading, a button, and a footer. Without a factory, every feature would copy and paste about 20 lines of HTML.

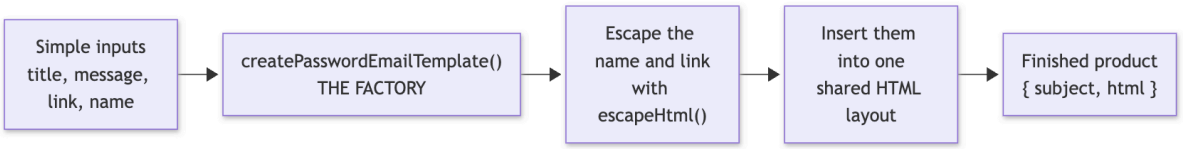
How TrackBack Implements It

`createPasswordEmailTemplate` in `src/integrations/email/emailTemplates.ts` is a factory function:

- Input: 5 simple values (title, message, link, name, and whether it is a reset link).

- Output: a ready-to-send `{ subject, html }` object.

The HTML layout is written once, inside the factory, and every email reuses it.



Before and After

	Without Factory	With Factory
Code per email	About 20 lines of HTML	1 function call
Changing the brand color	Edit every email	Edit one place
Consistent look	Not guaranteed	Always

```
// FACTORY: simple inputs in, finished email out
const createPasswordEmailTemplate = (
  title: string,
  message: string,
  actionUrl?: string,
  recipientName = "Buddy",
  resetLink = false,
): EmailTemplate => ({
  subject: title,
  html: `
    <div style="background:#f4f7fb;padding:32px;font-family:Arial,sans-serif;color:#172033">
      <div style="max-width:560px;margin:0 auto;background:#ffffff;padding:32px;border-radius:12px">
        <h1 style="margin:0 0 20px;font-size:24px">Trackback</h1>
        <p>Hi ${escapeHtml(recipientName)},</p>  <!-- name inserted safely -->
        <p>${message}</p>       <!-- message inserted -->
        ${actionUrl ? `<p style="margin:28px 0">
          <a href="${escapeHtml(actionUrl)}" style="background:#2563eb;color:#ffffff;padding:12px 20px;border-
            Continue
          </a>
        </p>` : ""}
        <p style="font-size:13px;color:#667085">${resetLink ? "This password reset link expires in 15 minutes.
      </div>
    </div>
  `,
});
```

5. Strategy Pattern (Behavioral Pattern)

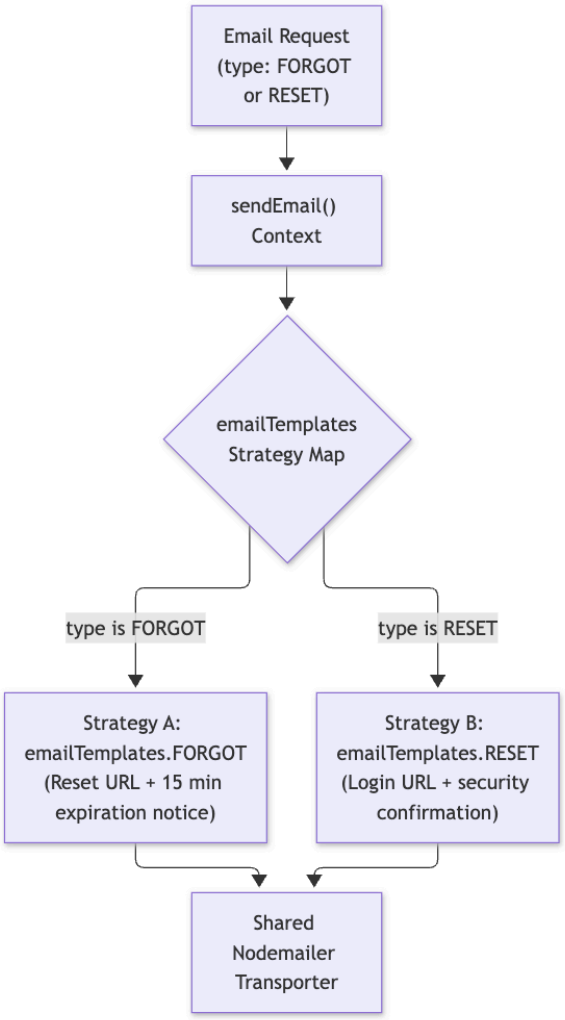
Core Idea: Define a family of interchangeable behaviors and pick the right one at runtime, without changing the code that uses them.

The Problem Without It: Inside the email sending service, we would write nested `if (type === 'FORGOT') ... else if (type === 'RESET') ...` statements. Adding new email types would require modifying, and risking bugs in, the core email sending logic.

How TrackBack Implements It:

TrackBack defines interchangeable email content strategies (`FORGOT` and `RESET`).

At runtime, `**sendEmail**` selects the required strategy based on `payload.type`, without changing how emails are sent over SMTP.



Code Implementation:

Strategy Definitions (src/integrations/email/emailTemplates.ts**) :**

```
const emailTemplates = {
  FORGOT: (resetUrl: string, recipientName?: string) =>
    createPasswordEmailTemplate(
      "Reset your Trackback password",
      "We received a request to reset your Trackback password. Click the button below to choose a new password
      resetUrl,
      recipientName,
      true, // Strategy parameter: shows 15-minute expiration notice
    ),
  RESET: (loginUrl: string, recipientName?: string) =>
    createPasswordEmailTemplate(
      "Your Trackback password was reset successfully",
      "Your Trackback password has been reset successfully. You can now sign in with your new password.",
      loginUrl,
      recipientName,
      false, // Strategy parameter: shows security contact notice
    ),
};

export default emailTemplates;
```

Executing Strategy at Runtime (src/integrations/email/emailService.ts**) :**

```
const sendEmail = async (payload: EmailTemplatePayLoad) => {
  const from = payload.senderEmail || process.env.EMAIL_FROM;

  // Dynamically selects strategy based on payload type
  const template = payload.type === "FORGOT"
    ? emailTemplates.FORGOT(payload.url, payload.recipientName)
    : emailTemplates.RESET(payload.url, payload.recipientName);

  // Common delivery mechanism
  await transporter.sendMail({
    from,
    to: payload.receiverEmail,
    subject: template.subject,
    html: template.html,
  });
};
```

Usage of sendEmail in the service folder:

```
await sendEmail({

  receiverEmail: user.email,

  recipientName: user.name,

  type: "FORGOT",

  url: resetUrl.toString(),

});
```

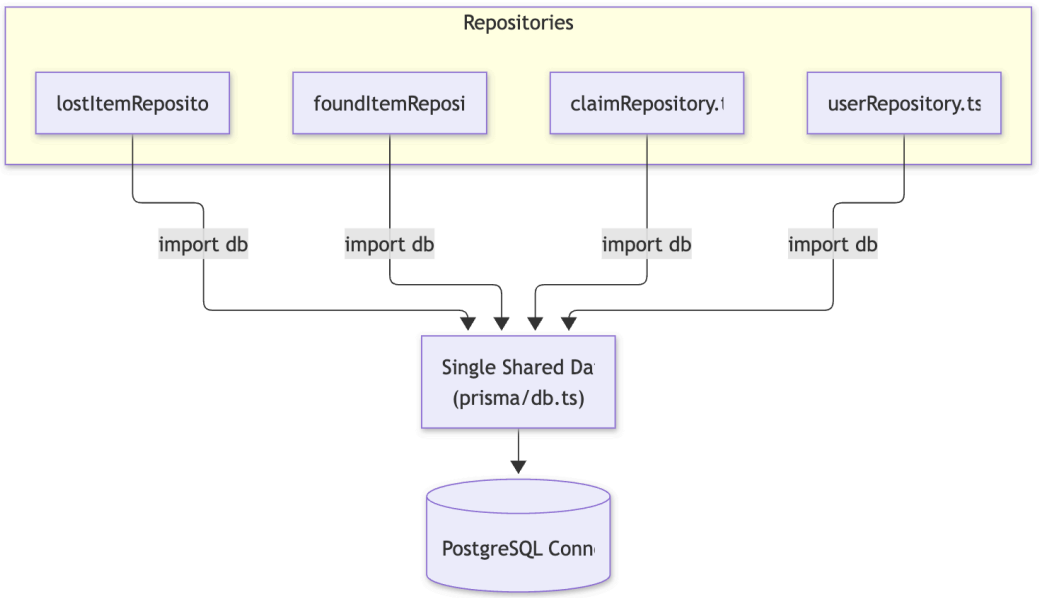
6. Singleton Pattern (Creational Pattern)

Core Idea: Create exactly one instance of an object and share it across the whole application.

The Problem Without It: Opening a new database connection or re-initializing the database driver on every API request is extremely heavy. It consumes CPU, wastes RAM, and can quickly crash PostgreSQL by hitting the maximum connection limit (`too many connections` error).

How TrackBack Implements It:

In `prisma/db.ts`, TrackBack creates a single database client instance. Because of Node.js ES Module caching, this instance is created once and shared across all repository files for the entire lifetime of the server.



Code Implementation:

```
import 'dotenv/config';
import postgres from '@prisma/orm-postgres/runtime';
import type { Contract } from './schema.d';
import contractJson from './schema.json' with { type: 'json' };

// SINGLETON: Instantiated once at startup, reused across all repositories
export const db = postgres<Contract>({
  contractJson,
  url: process.env['DATABASE_URL']!,
});
      You, 3 weeks ago • chore(init):setup project structure and docs
```

5. Operation

Example User Workflows

Workflow 1: Report Found Item

• User logs in → navigates to "Report Found Item"

• Fills form: title, description, category, location, date, photo

• System creates FoundItemPost with status=OPEN

• System runs matching and notifies matching lost-item owners

Workflow 2: Claim Found Item

• Owner sees notification: "Found item matches your lost item"

• Clicks → views found item → clicks "This is mine"

• Submits claim form with justification and proof photo

• System creates Claim with status=PENDING; Finder(found-item owner) is notified

Workflow 3: Review & Approve Claim

• Finder reviews claim details → clicks "Approve"

• Claim status → APPROVED; Found post → CLAIMED; Lost item → RECOVERED

• Owner notified: "Pick up at Lost & Found Office"

Workflow 4: Invite to Claim Request

• Finder see lost-item post -> clicks “I have this”

• Choose one of found-item post so that owner can make claim request to it

• Click “Notify Owner” -> Owner gets notified with “X believes they found your item Y” and can follow Workflow 2.

6. Conclusion

Project Overview

TrackBack is a well-architected, feature-complete Lost & Found application demonstrating professional software engineering:

• Separation of Concerns: Independent frontend modules; well-defined backend layers

• Design Patterns: A number of design patterns applied thoughtfully and consistently

• Scalability: Modular architecture supports growth and team expansion

• Maintainability: Clear structure, one-way dependencies, easy to change

• Security: JWT auth, HTTP-only cookies, input validation

• User Experience: Intuitive workflows for reporting, searching, claiming

Key Achievements

• Modular Frontend: Feature-based organization - Team members can work on different features in parallel with fewer merge conflicts, and dependencies between features stay explicit.

• Layered Backend: Organized into routes, controllers, services, and repositories.

• Intelligent Matching: Smart algorithm finds real matches

• Rich Notifications: Proactive alerts on matches, claims, approvals
• Clean API: 34 consistent endpoints with unified error handling

Future Enhancements

• Smarter Matching (v2): Location distance, temporal proximity, semantic similarity
• Machine Learning: Predict match quality from user feedback
• Real-Time Notifications: WebSocket support for instant alerts
• Admin Dashboard: Content moderation, analytics, user management
• Mobile App: React Native / Flutter version
• Campus SSO: Integration with institutional login systems

The project successfully demonstrates professional software engineering practices and delivers real value to the campus community.